基于 Spring Boot 的个性化推荐商城的设计与实现

摘要

随着信息化时代的发展，网上购物出现在人们的视野当中，其极大的满足了用户足不出户即可购买商品的需求，充实了人们的生活。如今的网上商城中的商品规模庞大，数量种类急速增长，针对用户的个性化推荐使得用户更便捷找到自己喜爱的商品，成为了迫在眉睫的需求。本文围绕商品信息过载的问题，设计并实现了一套基于 Spring Boot 的个性化推荐商城系统。系统在分析用户历史行为数据的基础上，引入矩阵分解模型，并结合特征增强与排序学习思想构建混合推荐模型，以提升推荐结果的准确性并缓解冷启动问题。通过前后端分离的设计，前端使用 Vue.js，后端使用 Spring boot

第 1 章 导论

选题目的及意义

随着信息化时代的发展，网上购物出现在人们的视野当中，其极大的满足了用户足不出户即可购买商品的需求，充实了人们的生活。如今的网上商城中的商品规模庞大，数量种类急速增长，针对用户的个性化推荐使得用户更便捷找到自己喜爱的商品，成为了迫在眉睫的需求

国内外研究现状

Amazon 是最早的电商商城，其早期从事图书销售的服务。直至今日已全面覆盖多久领域，Amazon 会通过对用户浏览记录，购买记录，搜索记录等行为。推测用户潜在喜欢的内容。在我国电商发展的大背景下，国内也兴起了诸如淘宝，京东等大型电子商城，他们大都采用了协同过滤、基于内容推荐、混合方法的应用。

推荐系统的研究起初是为了解决网络的发展所附带的数据过载的问题。1995 年，第一个个性化导航系统被提出。1997 年，最为成熟的协同过滤算法开始研究并使用。它易于实现，计算效率高，技术成熟等特点。它通过对用户历史行为或数据来收集用户偏好，从而找到相似的用户，也可以以物品为中心推荐相似的物品。或是基于模型根据评分数据得出模型推荐算法，通过机器学习和数据挖掘建模后根据模型进行预测计算

主要研究内容

网上商城是为了解决人们在网上消费的需求，提升用户体育，但是随着商品种类与数量的急剧增长，用户面对海量数据时难免头昏眼花。无法获得自己真正想要的资源。因此如何帮助用户快速找到自己喜爱的商品，如何实现个性化推荐功能，如何提供高质量的推荐。是目前急待解决的问题。基于个性化推荐的商城解决了这个问题，通过结合机器学习，大数据以及加上成熟的前后端技术实现的

开发技巧，在理论和实践中都有着重要的意义。
推荐算法设计直接关乎推荐性能的好坏。我研究了推荐系统和商城的现状，确定了本文的现实意义。通过技术选型和理论分析，为系统的实现提供了理论基础。通过运用混合推荐设计解决了商城系统中冷启动的问题，使得这一常见问题得到解决

论文组织结构

第 2 章 相关技术选型与理论基础

推荐算法概述

传统的协同过滤算法

协同过滤算法是推荐系统中应用最广泛的算法，他基于用户对物品的历史评分数据，从而提取用户或物品的潜向量，构造用户和物品间的相似矩阵，从而获得推荐。该算法忽略了潜在的信息，且排序结果的头部效应明显，处理稀疏矩阵的能力弱。

矩阵分解算法

矩阵分解算法将 m*n 维的矩阵及分解为 m*k 维用户矩阵和 k*n 维物品矩阵，m 是用户人数，n 是物品个数，k 是隐向量维数，维数越小，泛化能力越强，维数越大，潜向量表达性能越强，泛化能力越弱。

排序学习框架

深度学习基础

神经网络基本架构

激活函数的选择与作用

传统的矩阵分解只能捕捉线性关系，通过引入模型的非线性运算，能更真实的反应多层感知机的线性局限，从而在理论上使得大模型具备任意连续函数的能力。而激活函数对于模型的学习具有举足轻重的意义，通过激活函数进行的非线性映射真实的模拟了用户偏好的非线性特征

Sigmoid 函数

Sigmoid 函数将神经网络最后一层的线性输出映射至 区间，从而将推荐问题转换为二分预测问题，其数学公式如，所示：

$$\boldsymbol{\sigma(x) = \frac{1}{1 + e^{-x}}}$$

开发技术栈选型

JPA/Hibernate 持久层框架与数据库选型

Java ,Vue 与 Spring Boot 框架

java 是一门面向对象的高级编程语言，它拥有着通用性，简单性，可移植性，多线程等特点，他的口号是：一次编程，随意运行 意味着它可以在任何平台上运行而不需重编译。Springboot 是基于 SPring 4.0 开发的框架它通过一个叫 pom 的文件统一管理依赖包，从而免去繁杂的配置工作，并且他提前约定了程序结构，使开发人员只需关注业务逻辑。Vue.js 是一个前端框架，他的核心是数据驱动和组件化，他在 HTML 上运行 Vue 实例，Databindings 监控 Model 的数据，若变化则改变 DOM 上的数据。DOMlistener 检测 Dom 中的数据，若变化则更改 Model 中的数据

第 3 章 商城个性化推荐算法的设计与建模

本文围绕商品急剧增长的场景，针对推荐系统中常见的数据稀疏、预测精度不足以及冷启动等问题，对推荐算法进行整体设计。以用户历史行为为基础，引入排序学习，构建推荐模型整体框架，将非结构化或半结构化数据转换为可用于模型训练的结构化数据。然后将数据进行特征表示后作为输入进入推荐模型进行输出。

数据准备与预处理

原始数据收集及清洗

特征工程：特征构建与特征交叉

为了强化推荐模型的记忆能力和预测准确度，本研究在 wide 路径中使用了高阶交叉特征，其中，用户身份/属性与商品聚类标签（Cluster ID）的交叉是核心设计之一。这种设计能够捕捉到不同群体在特定场景下的偏好差异。例如，模型可以学习到“年轻女性群体（用户属性）”在面对“户外运动场景（商品聚类标签）”时，对特定品牌或价格区间的敏感度。

基于 Wide & Deep 的混合推荐模型

Wide & Deep 模型由 Google 提出,他与其他模型不同的特点是将线性模型记忆能力强的特点和神经网络便于泛化的特点结合起来，把单输入层的 Wide 部分与由 Embedding 层和多隐层组成的 Deep 部分连接起来 ，单层的 Wide 部分善于处理大量稀 疏的 id 类特征； Deep 部分利用神经网络表达能力强的特点，进行深层的特征交 叉 ， 将 wide 和 deep 层输出结果加权求和得到最终值

Wide 线性模型设计

Wide 部分仅输入 user_id、main_category 向量，通过对两个向量的交叉乘积，从而捕捉特定用户对于某个类别的特定偏爱，这样的记忆能力极强使得用户访问时能推荐他喜欢的商品

Wide 的数学表示如下：

$$\boldsymbol{\phi_k(x)} = \prod_{i=1}^{d} \boldsymbol{x_i^{c_{ki}}}, \quad \boldsymbol{c_{ki}} \in \{\mathbf{0}, \mathbf{1}\}$$

Deep 非线性模型设计

Embedding 层

热编码（One-hot Encoding）是处理类别特征的传统表征方式，通过将离散的商品类目映射到高维正交向量空间的单一维度来实现。，如一个商品属于电子类，则电子类的位为 1，其余全为 0 。One-hot 编码优势是实现简单，缺点是每增加一位维度加 1，容易维度爆炸，不能捕捉向量间的关系。为克服上述缺陷，本文使用 Embedding 技术将高维稀疏特征映射至低维稠密向量空间，它还能够捕捉到隐变量之间的非线性语义关联，为后续深度神经网络进行高阶特征交叉提供了信息丰富的输入。本系统针对 user_id 与 parent_asin 等离散 ID 特征构建了 Embedding 矩阵 。在模型前向传播过程中，通过查找表（Lookup Table）操作将原始索引映射为低维稠密向量 。这些向量不仅大幅降低特征空间维度，还能在向量空间中捕捉用户与商品之间潜在联关键。

多层感知机（MLP）

特征向量由 Embedding 层拼接而成，每一层隐藏层执行如下计算:

$$\boldsymbol{a^{(l+1)}} = \boldsymbol{\sigma}\left(\boldsymbol{W^{(l)}a^{(l)}} + \boldsymbol{b^{(l)}}\right)$$

其中， 表示第$\boldsymbol{W^{(l)}}$层的权重矩阵， $\boldsymbol{b^{(l)}}$为偏置项向量，$\boldsymbol{\sigma}$ 为非线性激活函数。为了提高模型的训练效率并缓解深层网络中常见的梯度消失问题，本研究采用 ReLU (Rectified Linear Unit) 作为激活函数。其数学表达式为：

$$\boldsymbol{f(x) = max(0, x)} = \begin{cases} \boldsymbol{x, x > 0} \\ \boldsymbol{0, x \leq 0} \end{cases}$$

矩阵分解与特征增强策略

矩阵分解是将物品-用户矩阵分解成两个向量的乘积

$$\boldsymbol{R_{(u,i)} = P_u * Q_i^T}$$

基于 K-means 的商品聚类 本研究首先利用商品隐含向量（由矩阵分解获得）作为输入，采用 K-means 聚类算法 将全量商品划分为 个不同的语义簇$\boldsymbol{SSE} = \sum_{j=1}^{K} \sum_{i \in C_j} \left|\left|\boldsymbol{Q_i} - \boldsymbol{\mu_j}\right|\right|_2^2$

特征融合与输入构建 聚类完成后，每个商品 将获得一个额外的离散特征 。我们将此 ID 进行 Embedding 转换后，与原始特征进行拼接。融合后的特征向量 修正为：$\boldsymbol{V_{final} = P_u \oplus Q_i \oplus E_{cluster} \oplus f_{num}}$

模型训练与排序策略

第 4 章 推荐商城的设计与实现

系统需求分析

随着商城数据量呈指数级增长，不仅用户面对海量商品无从下手，商城也需要精准快速的展示商品信息，本系统旨在实现一个满足基本的购物功能，并且能精确化推送的商城平台，用以解决推荐精确度和冷启动问题

用户与权限需求

对于用户管理服务，本系统在原有的账户生命管理的基础上，创新性的实现了在分布式环境下解决状态共享和差异化服务权限的问题，具体功能如下：
1.用户基础功能需求 系统不仅提供了完整的用户组测和登陆体系，在组测阶段需对用户提交的敏感信息进行加密处理保障数据库的安全性和完整性。登录阶段支持多维度身份验证以防止暴力破解。

商品与检索需求

现代商城的场景下，商品信息的展示效率直接影响了该产品对用户的转化率，在海量的数据下，本系统创新性的开发了多层级的检索与过滤体系，以满足不同场景下用户的需求。通过对关键词的模糊匹配能力以确保在用户输入模糊时仍能精准捕捉到商品，针对海量的数据，接口具备低延迟相应的特点，同时对高频搜索词进行缓存。为了降低用户的任职过在，系统基于商品的独有属性建立了分层分类体系。为减轻服务器压力以实现更好的展示控制，系统以高效的分页逻辑确保单次请求只加载固定数量的商品数据

个性化推荐需求

本系统的核心在于高度闭环自动化的个性化推荐系统。针对电商场景下的用户兴趣不明朗，数据稀疏度高，商品冷启动等挑战，系统需满足一下深层次的用户需求：

实时用户行为采集与多维度特征提取需求。系统需捕捉用户在平台上的所有互动，以及隐式反馈（浏览时长、点击页面、加入购物车）这些数据需要通过 Recommend Service 来进行持久化，并转换为模型可理解的特征向量

交易流转需求

电商系统的交易链路是整个数据流的关键，系统需要构建一套稳定、透明且有一定容错能力的交易体系，从前台选购到后端订单状态机变化的整个流程。
作为交易最开始的地方，购物车需提供高度灵活可靠的交互操作，为提升响应速度，系统需求支持前端利用 localStorage 进行临时存储的同时，当用户进行登陆操作后，系统需具备将 Cart 实体合并的功能从而确保数据据一致性。购物车需根据实时 Product Service 提供的单价、库存状态以及用户等级折扣来动态计算总金额

非功能性需求

为保证商城平台在网络环境下稳定运行，系统除了实现核心业务功能外，还需再并发处理，响应时延以及数据可靠性满足以下指标：

高并发处理能力

系统需利用 Spring Cloud Gateway 作为统一的流量入口，通过断路器和限流器对出现异常的流量进行拦截，从而防止底层为服务因过载而崩溃。

在分布式环境下，网关需配合注册中心动态感知微服务实例状态，并将请求均匀分发至多个后端节点来确保系统在大流量的情况仍然具有可靠性

推荐实时性

推荐性算法是本系统的核心特色，他的推理计算效率直接影响用户存留需求规定 Wide & Deep 模型的计算过程（包括特征转换、Embedding 查找及前向传播）必须在**百毫秒级（<200ms）**内完成。由于商品规模巨大，通过轻量级的过滤策略筛选出候选子集，再由神经网络进行重排序，从而确保前端 Vue 组件能实现秒级刷新从而维持流畅的用户交互体验。

数据一致性

在微服务架构中，一个交易逻辑往往跨越多个物理隔离的数据库（如订单库、支付库、库存库），必须可靠的解决分布式事务。当用户下单并支付成功后，系统需确保订单状态的更新（Order Service）、账户余额的扣减（User Service）以及商品库存的减少（Product Service）在逻辑上具备一致性。若支付流程发生异常，必须具备补偿机制（如事务消息或 TCC 模式）来回滚已冻结的库存，防止出现“超卖”或“账实不符”的现象，保障系统核心交易数据的绝对可靠。

系统架构设计

四层系统架构

用户接口层

用户接口层通常负责处理前端的请求并返回结果。Spring Cloud Gateway 网关负责路由分发、跨域处理（CORS）与身份鉴权。

应用层

负责对业务流程的编排。本层通常不包含核心业务规则，主要协调不同领域的服务完成业务任务 。如在推荐流程中，负责调用推荐算法接口并处理数据返回。

领域层

系统最核心的一层，包含核心业务实体与算法逻辑。本文设计的基于 Wide & Deep 的混合推荐模型、矩阵分解算法及 K-means 商品聚类均实现在此层 。

基础设施层

负责数据的持久化存储与技术支撑。利用 MyBatis 持久层框架连接 MySQL 数据库，实现用户行为数据与商品细节的持久化处理 。

数据库 E-R 模型设计

本系统遵循微服务架构的“一服务一数据库”原则将数据存储拆分为 7 个独立的逻辑库。本文是首先通过全局 E-R 模型展示系统的数据结构，随后对核心数据表进行详细定义

系统核心功能实现

商城核心功能实现

系统通过领域层定义的 BehaviorType 枚举类，涵盖了从隐性反馈（浏览、点击）到显性意图（加购、购买、评价）的全链路交互。每种行为被赋予不同的特征权重，作为推荐引擎的核心输入。设计 UserBehavior 实体用于持久化交互数据。字段包含 userId、productId、behaviorType 以及时间戳 createdAt。通过记录时间，模型能够捕捉用户兴趣的函数，从而提升推荐的时效性。

用户服务模块

系统采用 JPA/Hibernate 作为持久层框架 。User 实体类包含了用户的基础信息、账户余额及会员等级，其中 passwordHash 用于存储加密后的敏感信息，确保数据库安全性 。

```
@Data
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String username;

    private String email;
    private String passwordHash;

    @Enumerated(EnumType.STRING)
    private UserLevel level;

    private BigDecimal balance;

    public enum UserLevel {
        BRONZE, SILVER, GOLD, PLATINUM
    }
```

```
}
@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @GetMapping
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUserById(@PathVariable Long id) {
        return userService.getUserById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.createUser(user);
    }

    @PutMapping("/{id}")
    public ResponseEntity<User> updateUser(@PathVariable Long id,
@RequestBody User user) {
        return userService.updateUser(id, user)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
        return ResponseEntity.noContent().build();
    }
}
```

商品服务模块

Product 实体类映射了商品信息。系统通过 category 字段对商品进行逻辑划分，以便为后续推荐算法提供结构化的类目特征（Category ID）

```
@RestController
```

```
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    @GetMapping
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return productService.getProductById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping("/category/{category}")
    public List<Product> getProductsByCategory(@PathVariable
Product.Category category) {
        return productService.getProductsByCategory(category);
    }

    @GetMapping("/search")
    public List<Product> searchProducts(@RequestParam String keyword) {
        return productService.searchProducts(keyword);
    }

    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return productService.createProduct(product);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(@PathVariable Long id,
@RequestBody Product product) {
        return productService.updateProduct(id, product)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

```
@Data
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private BigDecimal price;
    private Integer stock;

    @Enumerated(EnumType.STRING)
    private Category category;

    private String description;

    public enum Category {
        ELECTRONICS, CLOTHING, HOME_GOODS, BOOKS, SPORTS
    }
}
```

订单服务模块

Payment 实体类记录了每一笔交易的元数据。系统引入了唯一交易流水号 transactionId，并记录了支付全生命周期的时间戳，以便进行财务对账与异常追踪 。

```
@RestController
@RequestMapping("/api/payments")
@RequiredArgsConstructor
public class PaymentController {

    private final PaymentService paymentService;

    @PostMapping
    public Payment createPayment(@RequestBody Payment payment) {
        return paymentService.createPayment(payment);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Payment> getPaymentById(@PathVariable Long id)
{
        return paymentService.getPaymentById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
```

```
        @GetMapping("/order/{orderId}")
        public ResponseEntity<Payment> getPaymentByOrderId(@PathVariable Long orderId) {
            return paymentService.getPaymentByOrderId(orderId)
                    .map(ResponseEntity::ok)
                    .orElse(ResponseEntity.notFound().build());
        }

        @GetMapping("/user/{userId}")
        public List<Payment> getPaymentsByUserId(@PathVariable Long userId) {
            return paymentService.getPaymentsByUserId(userId);
        }

        @PutMapping("/{id}/complete")
        public ResponseEntity<Payment> completePayment(@PathVariable Long id) {
            return paymentService.completePayment(id)
                    .map(ResponseEntity::ok)
                    .orElse(ResponseEntity.notFound().build());
        }

        @PutMapping("/{id}/refund")
        public ResponseEntity<Payment> refundPayment(@PathVariable Long id) {
            return paymentService.refundPayment(id)
                    .map(ResponseEntity::ok)
                    .orElse(ResponseEntity.notFound().build());
        }
    }
```

推荐服务模块实现

第 5 章 系统测试与结果分析

测试环境与指标说明

算法实验结果对比分析

系统功能与性能测试·

系统效果展示